

CI/CD による静的 Web サイトの自動デプロイ実践レポート

学籍番号: _____

氏名: 沼田 陽

2026 年 7 月 12 日

1 はじめに

本レポートでは、アプリケーションの更新時に自動でデプロイ作業が行われる仕組みである CI/CD (Continuous Integration と Continuous Delivery) を取り上げ、手動でのデプロイと自動デプロイを比較する。あわせて、今回 GitHub Actions と Azure Static Web Apps を用いて構築・公開した静的 Web サイトを題材に、CI/CD を導入することで運用がどのように改善されるかをまとめる。

2 CI/CD 導入前（手動でのデプロイ）

CI/CD が用いられていない場合、開発したファイルを開発者自身がサーバーへ配置して、はじめてユーザーが利用できる状態になる。講義の後半で扱った、VS Code から Azure App Service に対して「Deploy to Web App...」で ZIP を送信する操作がこれに該当し、変更のたびに人手で同じ配置作業を繰り返す必要がある。

2.1 課題

- **ヒューマンエラーの発生**：手作業でサーバーへ配置するため、操作ミスや手順の抜けが起こりやすい。
- **手間と時間がかかる**：プログラムを修正するたびに、ローカル (Mac など) から同じ内容を送り直す必要があり、更新回数が増えるほど負担が大きくなる。

3 CI/CD 導入後（自動デプロイ）

CI/CD を導入すると、コードの変更をリポジトリへ反映するだけで、ビルドから公開までが自動で実行される。本課題では、ローカルで編集した HTML / CSS を GitHub の main ブランチへ push するだけで、GitHub Actions が起動し、Azure Static Web Apps へファイルが自動でアップロードされる仕組みを構築した。おおよそ 1 分で公開 URL に反映され、手動でのアップロードは一切不要となる。

3.1 メリット

- **ヒューマンエラーの回避**：配置作業が自動化されるため、手作業に由来するミスを防ぐことができる。
- **開発に集中できる**：一度パイプラインを用意すれば、以降はサーバーへの配置を意識することなく、コンテンツの改善に専念できる (PaaS のメリットとも共通する)。

4 今回の構築と動作確認

リポジトリの直下に `index.html` と `style.css` を置き、`.github/workflows/` にデプロイ用の YAML を配置した。トリガーは `main` への `push` と Pull Request で、PR の際にはマージ前に内容を確認できるレビュー環境が自動生成される。`push` のたびに GitHub Actions の実行履歴が記録され、成功時にはログの末尾へ公開 URL が出力される（付録 図 1）。実際に、見出しの文字色を水色から赤へ変更するなどの修正をコミットして `push` しただけで、本番サイトへ即座に反映されることを確認した（付録 図 2、図 3）。

なお運用の過程で、デプロイ用ワークフローが手動作成分と自動生成成分の 2 本存在し、同じ Static Web App へ同時にデプロイして衝突するため、`push` のたびに一方が失敗していることに気づいた。デプロイ経路を 1 本に整理することの重要性も、本課題を通じて学べた点である。

5 おわりに

本課題を通じて、手動でのデプロイと自動デプロイの双方を理解した。手動でのデプロイは変更のたびに人手の配置が必要でミスの温床になりやすいが、GitHub Actions による CI/CD を用いることで、`push` するだけで安全かつ短時間にプログラムを公開できることを、実際の構築を通して確認できた。

6 付録

6.1 URL

今回構築した静的 Web サイトの公開 URL は以下の通りである（2026 年 7 月 12 日 時点で稼働を確認済み、HTTP 200 OK）。

`https://orange-stone-0cfe86d00.7.azurestaticapps.net`

※ ドメインには `.7.` が含まれる。これを省いた URL は 404 になるため、上記の完全な URL でアクセスすること。

6.2 関連スクリーンショット

構築・デプロイにあたるスクリーンショットを図 1～図 3 に示す。

【ここに図を貼付】 *GitHub* の *Actions* タブ、またはデプロイ成功時の画面
(<https://github.com/harukiti82/my-static-html-site/actions>)

図 1 デプロイ成功時の画面（GitHub Actions の実行履歴）

【ここに図を貼付】 *VS Code* の「ソース管理 (変更)」パネル, または *GitHub* のコミット差分 (緑=追加・赤=削除) の画面

図 2 変更の様子 (コミット・差分の画面)

【ここに図を貼付】 上記 *URL* をブラウザで開き, アドレスバーの *URL* とページ表示 (見出し「*Azure Static Web Apps Demo*」) が見える画面

図 3 公開されたサイトの表示 (現在の *URL*)